الفصل الثاني

الإطار النظري

2-1 مقدمة :

فرض التطور المتسارع في تقنيات المعلومات واقعاً جديداً اعتمدت فيه المؤسسات التعليمية بشكل رئيس على الأنظمة البرمجية لإدارة عملياتها الأكاديمية والترتيبية؛ لا سيما تلك التي تتطلب تنظيماً حثيثاً للموارد والبيانات. وتُعد عملية إعداد الجداول الدراسية وإدارة القاعات من أكثر المهام تعقيداً وأهمية؛ نظراً لتداخل عوامل متعددة تتمثل في المقررات، وأعضاء هيئة التدريس، والطلاب، والمساحات المكانية، والشرائح الزمنية، فضلاً عن حتمية الالتزام بمجموعة صارمة من القيود التنظيمية. وفي هذا السياق، تؤكد الأدبيات الأكاديمية أن مشكلة الجدولة الجامعية تندرج ضمن مسائل التحسين التوافقي (Combinatorial Optimization)المعقدة، والتي يتضخم التحدي فيها كلما اتسعت رقعة البيانات وتعددت التشابكات والقيود.

واستناداً إلى ذلك، يسلط هذا الفصل الضوء على المفاهيم الجوهرية المرتبطة بموضوع البحث، بدءاً من تحديد عنوان الدراسة ودلالات مصطلحاته الأساسية، مروراً بالتعريف بأبعاد الجدولة الأكاديمية وآليات حجز القاعات والتغلب على التعارضات الناجمة عنها. كما يناقش الفصل مفهوم الأنظمة الذكية ودورها المحوري في رفع كفاءة الجدولة، وصولاً إلى استعراض الحزمة التقنية والبرمجية المُعتمدة في بناء النظام وتطويره، والتي تشمل لغة **(Go)**، وإطار عمل **(Laravel)**، ومكتبة **(React)**، ونظام إدارة قواعد البيانات .**(MySQL)**

2-2 تصميم الانظمة[1] :

يُقصد بتصميم الأنظمة ذلك الفن الهندسي المعني ببناء بيئات برمجية متكاملة وقادرة على النمو والتكيف مع زيادة الأحمال، والصمود أمام التحديات التشغيلية على غرار المعماريات السحابية

[1]https://ae.linkedin.com/pulse/introduction-system-design-rohit-aggarwal-kzj8c?tl=ar

الكبرى في شركات مثل (Google و Amazonو Netflix) وتتجاوز هذه العملية حدود كتابة الشفرة البرمجية (Source Code) لتشمل التخطيط للهيكل العام، والربط بين مكونات البنية التحتية، وتنسيق الخدمات المتعددة(Services Orchestration) ، لضمان أعلى مستويات الأداء والاستقرار.

2-2-1 المنهجية الذهبية للتفكير والتصميم (منهجيةABCD):

تُعد منهجية (ABCD)من الأطر الفكرية والهندسية المعتمدة في تحليلات تصميم الأنظمة؛ حيث توفر إطاراً منطقياً للتحول من مرحلة الفهم الأولي للمشكلة إلى صياغة معمارية برمجية متكاملة ومستدامة:

A اسأل أسئلة جيدة(Ask) : تقتضي هذه الخطوة الاستكشاف الدقيق والاستخلاص الشامل لكافة المتطلبات (Functional & Non-Functional Requirements) الوظيفية و غير الوظيفية إلى جانب تحديد القيود التشغيلية والحدود التقنية للنظام قبل البدء في مراحل النمذجة أو كتابة الشفرات البرمجية.

B تجنب الكلمات الرنانة(No Buzzwords) : تتطلب الهندسة البرمجية الرشيدة اتخاذ القرارات التقنية بناءً على الحاجة الفعلية والجدوى الفنية للدراسة، وتجنب الانسياق خلف الأطر والتقنيات المستحدثة لمجرد شيوعها الإعلامي دون مسوغ تشغيلي واضح ومبرر.

C تفكير واضح ومنظم(Clear) : يقوم هذا المحور على التدرج الهيكلي المنظم، بدءاً من صياغة المخطط الهندسي العام (High-Level Architecture) وصولاً إلى تفكيك النظام إلى مكوناته الفرعية، مع تحديد نقاط الاختناق المحتملة (Bottlenecks) وضمان كفاءة تدفق البيانات.

D قيادة النقاش(Drive) : تتجسد هذه الركيزة في إدارة الحوار التقني بفاعلية، واقتراح الحلول والمعماريات البديلة، مع إجراء دراسات المفاضلة والموازنة الواعية (Trade-offs) بين معايير الأداء، والتكلفة، والتوفر العالي للخدمة، بما يضمن استقرار النظام ومرونته.

2-2-2 خصائص ومكونات تصميم النظام:

- **المعايير الأساسية** :يوازن التصميم الجيد بين الأداء (سرعة الاستجابة)، التوفر العالي (Up-time)، قابلية التوسع، أمان البيانات، وفعالية التكلفة.
- **النواء الأساسية** :يعتمد النظام على أدوات متخصصة مثل موازنات التحميل Load) (Balancers، الذاكرة المؤقتة) مثل(Redis ، قواعد البيانات(SQL/NoSQL) ، طوابير الرسائل) مثل(Kafka ، وأنظمة المراقبة والسجلات.

2-2-3 أنماط المعمارية الرئيسية للتصميم (Architecture Patterns)

- **العميل-الخادم :(Client-Server)** النموذج التقليدي لتطبيقات الويب والمحمول.
- **القائم على الأحداث :(Event-Driven)** معالجة غير تزامنية تناسب الإشعارات والتحليلات اللحظية.
- **النواة الدقيقة :(Microkernel)** نظام أساسي يعتمد على إضافات قابلة للتوسيع) مثل بيئات البرمجة.(IDEs
- **الخدمات المصغرة :(Microservices)** تقسيم التطبيقات إلى خدمات مستقلة تناسب الأنظمة الضخمة والموزعة. النظام الإلكتروني هو مجموعة متكاملة من المكونات المادية (دارات، أجهزة تحسس، معالجات) أو البرمجية التي تتفاعل معاً لمعالجة الإشارات والبيانات وتحويل المدخلات إلى مخرجات محددة.

2-2-4 تصميم واجهة وتجربة المستخدم (UI/UX)[2]:

[2] https://academy.hsoub.com/design/user-experience/%D8%AA%D8%B5%D9%85%D9%8A%D9%85-%D9%88%D8%A7%D8%AC%D9%87%D8%A9-%D8%A7%D9%84%D9%85%D8%B3%D8%AA%D8%AE%D8%AF%D9%85-%D9%88%D8%AA%D8%AC%D8%B1%D8%A8%D8%A9-%D8%A7%D9%84%D9%85%D8%B3%D8%AA%D8%AE%D8%AF%D9%85-ui-ux-r831/

هو المجال المختص ببناء منتجات رقمية (مواقع وتطبيقات) تجمع بين الجمال البصري وسهولة الاستخدام لتلبية احتياجات المستخدم بفعالية.

الفرق بين UX و UI:

تجربة المستخدم (UX – User Experience): تركز على منطق العمل ورعاية رحلة المستخدم، وسلاسة الانتقال بين الشاشات، وهيكلة المعلومات لضمان حل مشكلة المستخدم بأقل جهد ممكن.

واجهة المستخدم (UI – User Interface): تركز على المظهر البصري والتفاعلي، مثل اختيار نظام الألوان، أنواع الخطوط، تصميم الأزرار، الأيقونات، واتساق العناصر الشكلية.

2-2-5 مراحل تصميم المنتج الرقمي:

البحث والتحليل (Research): دراسة سلوك المستخدمين، تحديد المشكلات، وتحليل التطبيقات المنافسة.

معمارية المعلومات وهيكلة الشاشات (Wireframing): رسم مخططات سلكية مبسطة (Low-Fidelity) لتحديد التدفق ومواقع العناصر الأساسية.

التصميم البصري والتحريك (Prototyping): تحويل المخططات إلى واجهات عالية الدقة (High-Fidelity) وإضافة التفاعلات والحركات للتأكد من المظهر والوظيفة.

الاختبار والتحسين (Testing): تجربة النموذج مع مستخدمين حقيقيين لاكتشاف نقاط التعثر وتعديلها قبل تسليم المشروع للمطورين.

2-2-6 الأدوات المستخدمة في التصميم :

Figma: الأداة المعتمدة عالمياً للتصميم التفاعلي والتعاون المباشر بين الفرق.

Sketch & Adobe XD: أدوات متخصصة في بناء الواجهات والنماذج التفاعلية.

Balsamiq: أداة سريعة لبناء الهياكل السلكية المبدئية.

2-2-7 مبادئ تصميم الواجهات الناجح :

الاتساق (Consistency): استخدام لغة بصرية موحدة للأزرار والخطوط عبر كامل التطبيق.

إمكانية الوصول (Accessibility): مراعاة تباين الألوان وأحجام النصوص لتناسب جميع المستخدمين.

التغذية الراجعة (Feedback): إشعار المستخدم بنتيجة كل فعل يقوم به (مثل ظهور مؤشر التحميل أو تغيير لون الزر عند النقر).

2-3 الانظمة الالكترونية[3]:

النظام الإلكتروني هو مجموعة متكاملة من المكونات المادية أو البرمجية التي تتفاعل معاً لمعالجة الإشارات والبيانات وتحويل المدخلات إلى مخرجات محددة.

2-3-1 المكونات الأساسية لأي نظام إلكتروني:

- **المدخلات (Inputs)**: أجهزة تحسس (Sensors) أو أدوات إدخال تلتقط الإشارات الفيزيائية (كالحديث، الحرارة، أو الضغط) أو الرقمية.
- **المعالجة (Processing)**: دارات أو معالجات دقيقة (CPUs / Microcontrollers) تتولى توجيه البيانات، تحليلها، وإجراء الحسابات المنطقية.
- **المخرجات (Outputs)**: مشغلات (Actuators)، شاشات، أو مكبرات صوت تُحوّل نتائج المعالجة إلى أفعال ملموسة أو بيانات مرئية.

[3] https://www.scribd.com/document/1062883359/%D9%85%D8%A8%D8%A7%D8%AF%D9%89-%D8%A7%D9%84%D9%86%D8%B8%D9%85-%D8%A7%D9%84%D8%A7%D9%84%D9%83%D8%AA%D8%B1%D9%88%D9%86%D9%8A%D8%

2-3-2 تصنيف وأنواع الأنظمة الإلكترونية[4]:

2-3-2-1 حسب طبيعة الإشارة (Signal Processing):

- **الأنظمة التناظرية (Analog Systems):** تتعامل مع إشارات مستمرة متغيرة في الزمن، مثل مكبرات الصوت الصوتية (Audio Amplifiers) وأجهزة أجهزة الراديو التقليدية.
- **الأنظمة الرقمية (Digital Systems):** تتعامل مع إشارات مقطعة على شكل قيم ثنائية (0 و 1). تتميز بدقة عالية وسهولة في معالجة البيانات وتخزينها، مثل الحواسيب والهواتف الذكية.
- **أنظمة الإشارة المختلطة (Mixed-Signal Systems):** تجمع بين الدارات التناظرية والرقمية على شريحة واحدة لمعالجة إشارات العالم الحقيقي، مثل محولات الصوت (ADC/DAC) وأجهزة الاتصالات الحديثة.

2-3-2-2 حسب المجال والتطبيق (Application Domain):

- **الأنظمة المدمجة (Embedded Systems):** أنظمة صغيرة مخصصة لتنفيذ وظيفة محددة داخل جهاز أكبر، وتعتمد على متحكم دقيق (Microcontroller). تُستخدم في السيارات (أنظمة المكابح ABS)، الأجهزة المنزلية، والساعات الذكية.
- **أنظمة إلكترونيات القوة (Power Electronics Systems):** تُعنى بتحويل والتحكم في الطاقة الكهربائية ذات الجهود والتيارات العالية، مثل محولات الطاقة (Inverters) ومشغلات المحركات في السيارات الكهربائية.

---

[4] https://www.scribd.com/document/806288057/%D8%A3%D9%86%D9%88%D8%A7%D8%B9-%D8%A7%D9%84%D8%A3%D9%86%D8%B8%D9%85%D8%A9-%D8%A7%D9%84%D8%B1%D9%82%D9%85%D9%8A%D8%A9-%D9%88-%D9%81%D8%A7%D8%A6%D8%AF%D8%AA%D9%87%D8%A7

- **أنظمة الاتصالات (Communication Systems)**: تُستخدم لنقل البيانات والمعلومات عبر وسائط سلكية أو لاسلكية، مثل شبكات الألياف الضوئية، شبكات المحمول (G5)، وأنظمة الأقمار الصناعية.
- **أنظمة التحكم والأتمتة (Industrial Automation & Control)**: تُستخدم في المصانع لإدارة آلات الإنتاج والعمليات المعقدة تلقائياً باستخدام المتحكمات المنطقية القابلة للبرمجة (PLC) والروبوتات.
- **أنظمة المعلومات البرمجية (Information Electronic Systems)**: المنصات والأنظمة البرمجية التي تعمل فوق البنية التحتية الإلكترونية لإدارة البيانات المؤسسية وقواعد البيانات والخدمات السحابية.

2-3-3 استخدامات الانظمه الالكترونية[5]:

1 .الشركات والمؤسسات التجارية

تستخدم في إدارة الحسابات والمبيعات والمخزون.

2 .البنوك

تستخدم في إدارة الحسابات المالية والتحويلات البنكية.

3 .المدارس والجامعات

تستخدم في إدارة الطلاب والامتحانات والنتائج.

4 .المستشفيات

تستخدم في إدارة السجلات الطبية للمرضى.

5 .المؤسسات الحكومية

[5] https://yementk.com/blog/erp-systems/

تستخدم في تقديم الخدمات الإلكترونية للمواطنين.

2-3-4 الانظمه الادارية [6]:

تُشكل الأنظمة الإدارية المؤسسية البنية التحتية البرمجية والتنظيمية التي تحكم التدفق الكلي للبيانات والعمليات داخل المنظمة، حيث تمثل القاعدة الأساسية التي تُبنى عليها جميع تطبيقات ادارة الموارد الخاصة بالمؤسسة و الجدولة وحجز الموارد.

كما يرتبط النجاح التشغيلي للأنظمة الإدارية بمستوى تكاملها الشبكي مع المحركات البرمجية الأخرى في المؤسسة، وعلى رأسها أنظمة إدارة معلومات الطلاب(SIS) ، وأنظمة إدارة التعلم(LMS) ، وأنظمة تخطيط الموارد المؤسسية .(ERP) يتيح هذا التكامل التدفقي تغذية قاعدة البيانات المركزية بالبيانات المصنفة المحدثة لحظياً، مما يلغي التكرار الناجم عن إدخال البيانات يدوياً عبر أدوات منفصلة لا تتواصل فيما بينها .إن غياب هذا التكامل في الأنظمة القديمة يترتب عليه تعارض في السجلات، وتأخير في تحديث بيانات الشغور، وإعادة إدخال البيانات يدوياً لعدة ساعات، بينما يوفر الربط التلقائي واجهة موحدة تحدث كافة البيانات المؤسسية فور وصول المدخلات للنظام . تستوجب إدارة الأنظمة الإدارية إطاراً أمنياً محكماً يعتمد على التحكم في الوصول القائم على الأدوار لضمان سلامة البيانات وسريتها (Role-Based Access Control – RBAC) .يتم من خلال هذا الإطار توزيع الأدوار والامتيازات بحذر؛ حيث يُمنح مديرو النظام صلاحيات كاملة لإدخال البيانات، وتعديل المخططات الهيكلية، وتشغيل الخوارزميات ، بينما تقتصر صلاحيات الأقسام الأكاديمية على مدخلات مخصصة لترتيب المقررات والمحاضرين .وفي المقابل، يُتاح للطلاب وأطراف المجتمع المؤسسي الآخرين أدوار عرض مخصصة تفاعلية تتيح الاستعلام عن الجداول والحجوزات دون امتلاك صلاحية التعديل الهيكلي.

---

[6] https://www.annajah.net/%D9%86%D8%B8%D9%85-%D8%A7%D9%84%D9%85%D8%B9%D9%84%D9%88%D9%85%D8%A7%D8%AA-%D8%A7%D9%84%D8%A5%D8%AF%D8%A7%D8%B1%D9%8A%D8%A9-%D9%88%D8%A3%D9%86%D9%88%D8%A7%D8%B9%D9%87%D8%A7-article-37402

2-4 إدارة الحجوزات[87]:

تُعنى إدارة الحجوزات بالتأطير التنفيذي لعملية تخصيص الأصول المكانية والخدمية لفترات زمنية محددة ولجهات طالبة معينة، بهدف تعظيم الاستفادة التشغيلية منعاً للهدر. وتتضمن تنظيم عمليات الطلب والتخصيص للموارد الفيزيائية القابلة للاستهلاك الزمني؛ كالصالون، والمدرجات، والآلات، والقاعات التفاعلية .تعتمد الديناميكية الإدارية الحديثة على الربط بين السعة الاستيعابية للموارد المتاحة والطلب الفعلي المتغير، حيث تُسهم إدارة المواعيد المنظمة في تقليل الفترات الزمنية الضائعة وزيادة معدل الاستفادة من المنشآت .تتطلب العملية الموازنة بين الاستغلال الأقصى للمكان والحفاظ على جودة الخدمة عبر تجنب الاكتظاظ وتضارب المواعيد. ترتبط كفاءة نظام الحجز بمدى صرامة السياسات التنظيمية التي تعالج الإلغاءات ووصول المستخدمين .تُعد ظاهرة (No-Show)عدم الظهور من أكبر التحديات التي تؤدي إلى تجميد الموارد دون استغلال .لذلك، تدمج الأنظمة المتقدمة سياسات التحرير الآلي للمورد(Automatic No-Show Release) ؛ حيث يُلغى الحجز تلقائياً ويُتاح للمستخدمين الآخرين إذا لم يسجل المستفيد دخوله الفعلي أو يحضر خلال نافذة زمنية محددة من بداية موعد الحجز.

2-4-1 إدارة الحجوزات إلكترونياً [109]:

يُمثل التحول إلى إدارة الحجوزات إلكترونياً الانتقال من المعاملات الورقية المحدودة إلى منظومة رقمية متكاملة تعمل على مدار الساعة وتستجيب للتغيرات لحظياً.

2-4-2 آليات الحجز الفوري والمزامنة السحابية:

---

[7] https://www.linkin1.com/blog/%D8%A8%D8%B1%D9%86%D8%A7%D9%85%D8%AC-%D8%AD%D8%AC%D9%88%D8%B2%D8%A7%D8%AA-%D9%88%D9%86%D8%B8%D8%A7%D9%85-%D8%A5%D8%AF%D8%A7%D8%B1%D8%A9-%D8%A7%D9%84%D8%AD%D8%AC%D9%88%D8%B2%D8%A7%D8%AA-%D9%88%D8%A7%D9%84%D9%85%D9%88%D8%A7%D8%B9%D9%8A%D8%AF

[8] https://www.scirp.org/reference/referencespapers?referenceid=3602735

[9] https://arxiv.org/abs/2412.06874?utm_source

[10]https://iaeme.com/MasterAdmin/Journal_uploads/IJRCAIT/VOLUME_8_ISSUE_1/IJRCAIT_08_01_245.pdf

تعتمد إدارة الحجوزات إلكترونياً على (Real-time Cloud Synchronization) التحديث التزامني السحابي اللحظي ، حيث يتم تسجيل أي حجز يُجرى عبر الإنترنت في قاعدة البيانات المركزية وملاحظته عبر جميع واجهات المنظومة فوراً .يلغي هذا التحديث التلقائي الفوري احتمالات التعارض والحجز المزدوج، كما يمنح المشرفين قدرة كاملة على متابعة حالة توفر القاعات والمواعيد لحظة بلحظة.

2-4-2-1 أدوات التكامل مع البرمجيات والواجهات الرقمية :

تتيح المنصات الإلكترونية الحديثة دمج أدوات الحجز (Booking Widgets) مباشرة داخل المواقع الإلكترونية المؤسسية، أو مشاركتها عبر روابط مباشرة، أو إتاحتها عبر تطبيقات الهواتف المحمولة والمنصات الرقمية المختلفة .كما تدعم هذه الأدوات إمكانية تحويل رابط الحجز إلى زر تفاعلي مثل "احجز الآن" عبر الوسائط المتعددة، مما يتيح للمستخدمين استعراض المواعيد المتاحة وتأكيد حجوزاتهم في ثوانٍ معدودة دون حاجة لخلفية تقنية.

2-4-2-2 الأتمتة المتقدمة للخدمات والمعاملات:

تتجاوز الحجوزات الإلكترونية مجرد تسجيل الموعد لتشمل أتمتة الخدمة كاملة؛ كتعيين الموظف أو الكادر المناسب للخدمة تلقائياً، وتحديد مدة الخدمة، وضبط التسعير سواء كانت الخدمة بمقابل أو استشارة مجانية .تضمن المنظومة كذلك إرسال رسائل تأكيد وتذكير عبر البريد الإلكتروني والرسائل النصية، مع إتاحة خيارات تعديل المواعيد أو إلغائها ذاتياً من قبل العملاء والمستخدمين وفق القواعد المقررة.

2-4-3 أنظمة إدارة الحجوزات [11]:

[11] **طارق عبد المجيد .(2021) .**تطوير أنظمة إدارة الموارد والحجوزات المؤسسية باستخدام المعماريات البرمجية متعددة الطبقات .*المجلة الدولية لعلوم الحاسب وتكنولوجيا المعلومات*، مجلد 12، عدد 3، ص ص 45–60

مثل أنظمة إدارة الحجوزات (Booking Management Systems) البيئة البرمجية الشاملة التي تجمع بين واجهات المستخدم، ومحركات الحساب الخوارزمية، وأدوات تحليل البيانات لتوفير بيئة تشغيل متكاملة.

2-4-3-1 المكونات البرمجية وهندسة الأنظمة الحديثة :

تُبنى أنظمة إدارة الحجوزات الحديثة وفق هندسة برمجية متكاملة الطبقات (Full-Stack Architecture). تتكون الطبقة الأمامية (Frontend) من واجهات تفاعلية مرنة تُصمم باستخدام تقنيات حديثة مثل Vue.js و Tailwind CSS لتسهيل تجربة الاستخدام. أما الطبقة الخلفية (Backend)، فتُبنى باستخدام أطر عمل متطورة مثل Python و Flask لمعالجة المنطق التشغيلي، والربط مع قاعدة بيانات علائقية مثل PostgreSQL لتخزين السجلات. تضمن هذه الهندسة استقرار النظام، وسرعة معالجة الطلبات المتزامنة، وسهولة التوسع المستقبلي لتلبية الاحتياجات المؤسسية المتزايدة.

2-4-3-2 المراقبة التزامنية واستغلال الطاقة الاستيعابية:

تتميز الأنظمة الحديثة بالقدرة على ربط النشاط المخطط بالاستغلال الفعلي للمساحات (Actual Occupancy). توفر هذه الأنظمة أدوات تحليليّة تُظهر معدلات استخدام المنشآت والمساحات عبر الزمن، مما يساعد الإدارة على اكتشاف القاعات والموارد المهدرة وإعادة توزيع الفعاليات والمحاضرات عليها لرفع كفاءة المنشآت واستعادة المساحات الضائعة.

2-4-3-3 إدارة التوزيع التنافسي وتحديد الأولويات :

عند حدوث تزامن في طلب الموارد المحدودة من قبل أطراف متعددة، تعتمد أنظمة إدارة الحجوزات على محركات أولويات مبرمجة تقيم وزنية كل طلب بناءً على القواعد التنظيمية للمؤسسة. يتم معالجة التعارضات تلقائياً عبر ترتيب الطلبات وفق الأهمية التشغيلية، أو أولوية أسبقية الحجز، أو أهمية الفعالية الأكاديمية والمؤسسية.

2-5 إدارة حجز القاعات الدراسية [12]:

تُشكل القاعات الدراسية والمعامل والمدرجات الجامعة فئة خاصة من الموارد المؤسسية التي تتطلب إدارة حجز دقيقة تجمع بين متطلبات الجداول الفصلية والطلبات العارضة.

2-5-1 تصنيف القاعات الدراسية والموارد الأكاديمية:

تتطلب الإدارة الفعالة للقاعات الدراسية إنتاج قاعدة بيانات مخصصة تصنف القاعات بناءً على طاقتها الاستيعابية، والتجهيزات التقنية المتاحة بها (مثل أجهزة العرض، والأنظمة الصوتية، والتجهيزات المعملية)، ونوعية النشاط المسموح به .يتيح هذا التصنيف الدقيق تلبية الاحتياجات التعليمية الخاصة؛ مثل متطلبات التدريس المشترك (Co-teaching) أو الدروس التي تتطلب تقسيم الطلاب إلى مجموعات فرعية، وهي متطلبات تفشل أنظمة الحجز التقليدية في معالجتها لعدم قدرتها على التكيف مع الموارد المركبة.

2-6 دورة حياة تطوير الأنظمة (System Development Life Cycle - SDLC)[1413]:

تمر البرمجيات والأنظمة المعلوماتية عمومًا أثناء تطويرها بأربعة مراحل رئيسة وفي كل مرحلة مجموعة من الخطوات. ينتج عن كل مرحلة مجموعة من المخرجات تُبنى عليها مراحل لاحقة هذه الأربعة مراحل متتابعة ومترابطة تحاكي المنطق الهندسي للحلول التقنية، وهي: التخطيط(Planning) ، والتحليل(Analysis) ، والتصميم(Design) ، والتنفيذ .(Implementation) تتجاوز هذه المراحل كونها مجرد خطوات تنفيذية متتالية؛ إذ تمثل منظومة متكاملة تخضع للمواصفات القياسية الدولية المنظمة لعمليات هندسة البرمجيات.

---

[12] **حامد، أحمد عبد الفتاح .(2018)** *بحوث العمليات وتطبيقاتها في إدارة المؤسسات التعليمية* (الطبعة الأولى). دار الفكر العربي، القاهرة

[13] https://academy.hsoub.com/programming/general/%D8%AF%D9%88%D8%B1%D8%A9-%D8%AD%D9%8A%D8%A7%D8%A9-%D8%AA%D8%B7%D9%88%D9%8A%D8%B1-%D8%A7%D9%84%D8%A8%D8%B1%D9%85%D8%AC%D9%8A%D8%A7%D8%AA/

[14] https://www.wiley.com/en-sg/systems-analysis-and-design-8th-edition-p-9781119803782

التخطيطplanning :

تعتبر مرحلة التخطيط نقطة الانطلاق الحاسمة التي يتقرر فيها مدى تسويغ وبدء مشروع نظام المعلومات، وتُعنى بالإجابة عن التساؤل الجوهري حول أسباب بناء النظام والفوائد المرجوة منه. تبدأ هذه المرحلة عندما تقدم الجهة الراعية للمشروع ما يُعرف بطلب النظام (System Request)، وهو الوثيقة التأسيسية التي توضح المشكلة التجارية المراد حلها أو الفرصة الاستثمارية المتاحة، مع التحديد المبدئي للقيم المؤسسية المكتسبة والقيود المتوقعة. ولا يقتصر هذا الطلب على الطرح الفني، بل يمتد إلى التحقق من مدى المحاذاة الاستراتيجية بين الأهداف التقنية للمشروع والتوجهات الكبرى للمؤسسة، لضمان استغلال الموارد الفنية والمالية في مسارات تحقق مزايا تنافسية ملموسة.

عقب تحرير طلب النظام، يخضع المشروع المترقب لتقييم صلب يسمى دراسة الجدوى الشاملة، وهي دراسة متعددة الأبعاد تستهدف التحقق من قدرة المؤسسة على المضي قدماً في المشروع من ثلاثة محاور رئيسية[1]. المحور الأول هو الجدوى الفنية التي تقيس مدى التوافق التقني مع متطلبات المشروع، من حيث توافر الخبرات البشرية، واستيعاب مخاطر التقنيات الحديثة، والقدرة على التعامل مع حجم التعقيد البرمجي. والمحور الثاني هو الجدوى الاقتصادية التي تتضمن إجراء تحليل دقيق للتكاليف والمنافع التقديرية، وحساب مؤشرات الأداء المالي مثل عائد الاستثمار ، وصافي القيمة الحالية ، وتحليل نقطة التعادل لمعرفة المدى الزمني لاسترداد رأس المال واستدامة النظام مادياً. أما المحور الثالث فهو الجدوى التنظيمية والتشغيلية التي تقيس مدى تقبل البيئة المؤسسية للنظام الجديد، ومدى قدرة المستخدمين على استيعابه ضمن سير العمل اليومي، لتفادي المقاومة التنظيمية وتسهيل عمليات الاعتماد الميداني.

تُختتم مرحلة التخطيط بوضع إطار إدارة المشروع، حيث يقود مدير المشروع عملية إعداد خطة العمل الحاكمة التي توزع المهام وتحدد الموارد المادية والبشرية الجديرة بإنجاز كل مرحلة.

التحليلanalysis :

تنتقل دورة الحياة في مرحلتها الثانية إلى التحليل الفعلي، وهي المرحلة التي تُعنى بالإجابة التفصيلية عن أسئلة تحديد نطاق العمل: من سيتفاعل مع النظام؟ وما هي الوظائف والعمليات المطلوبة بدقة؟ وأين ومتى سيتحدد النطاق التشغيلي للنظام؟. تبدأ هذه المرحلة بالصياغة المباشرة لإستراتيجية التحليل، حيث يقوم محللو الأنظمة بدراسة النظام الحالي (As-Is System) للوقوف على الاختناقات التشغيلية، وعيوب الأداء، والتكاليف المخفية[1]. وتتضمن هذه العملية استخدام أدوات تحليل المشكلات، وتحليل السبب الجذر، واستكشاف مجالات التحسين لتشكيل رؤية متكاملة حول النظام المستقبلي.

تشكل عملية استنباط وجمع المتطلبات الأساس الجوهري لنجاح مرحلة التحليل، حيث يتعين على المحللين الانخراط المباشر مع مختلف طبقات أصحاب المصلحة. وتعتمد هذه العملية على مروحة واسعة من الأدوات والتقنيات المعيارية، مثل المقابلات الشخصية المفصلة، والاستبيانات الموجهة، وتطبيق جلسات التطوير الجانبي المشترك (Joint Application Development – JAD)، وتتبع الملاحظة المباشرة لسير العمل. وتتيح هذه الأدوات التمييز الدقيق بين المتطلبات الوظيفية التي تصف السلوك البرمجي والعمليات الحسابية للنظام، والمتطلبات غير الوظيفية التي تحدد خصائص الأداء، والأمان، والاعتمادية، وسهولة الاستخدام، وهو ما يتوافق مع أطر جودة البرمجيات.

تُترجم المتطلبات المجمعة بعد ذلك إلى نماذج مفهومية وهيكلية مجردة باستخدام أساليب التحليل الهيكلي أو أساليب التحليل موجه الكائنات (Object–Oriented Analysis – OOA). تتضمن نماذج العمليات إنشاء مخططات تدفق البيانات (Data Flow Diagrams – DFD) أو حالات الاستخدام (Use Cases) ومخططات الأنشطة ضمن لغة النمذجة الموحدة (UML) لوصف تفاعل الأطراف مع النظام وتدفق المعلومات عبر إجراءاته التشغيلية في النظام البرمجي. وفي المقابل، تتولى نماذج البيانات توثيق الكيانات والمعلومات الأساسية من خلال مخططات العلاقة بين الكيانات (Entity Relationship Diagrams – ERD) أو مخططات الفئات (Class Diagrams).

تتوج مرحلة التحليل بتجميع كافة التحليلات والمخططات في وثيقة معيارية شاملة تُعرف بـ "النظام المقترح " (System Proposal). تُقدم هذه الوثيقة إلى الإدارة التنفيذية وأصحاب القرار لتقييم مدى دقة الفهم المكتسب في استمرارية المشروع. وتلتزم هذه الوثيقة في هيكليتها وصياغتها بالمواصفات القياسية.

التصميمdesign :

تركز مرحلة التصميم على تحويل المتطلبات الوظيفية وغير الوظيفية المحددة في مقترح النظام إلى مواصفات ومخططات فنية هندسية دقيقة تشرح التفاصيل العملية لكيفية بناء النظام. تتطلب هذه المرحلة اتخاذ قرار استراتيجي أولي يتعلق بإستراتيجية اكتساب النظام؛ إذ يدرس المهندسين المفاضلة بين ثلاثة خيارات حاسمة: التطوير الداخلي المخصص (Custom Development) لتلبية احتياجات فريدة، أو شراء الحزم البرمجية الجاهزة (Commercial Off–The–Shelf – COTS) وتعديلها لتقليل التكلفة والوقت، أو الاستعانة بمصادر خارجية (Outsourcing) لتطوير النظام عبر شركات متخصصة.

عقب حسم إستراتيجية الاكتساب، يتولى مهندسو الأنظمة وضع التصميم المعماري وتصميم البنية التحتية، وهو الإطار الذي يحدد البيئة التقنية التي سيعمل ضمنها النظام، بما يشمل المكونات المادية، ونظم التشغيل، وشبكات الاتصال. ويتم في هذه الخطوة اختيار النمط المعماري المناسب مثل معمارية العميل/الخادم، أو المعمارية المعتمدة على الخدمات المصغرة (Microservices)، أو البيئات السحابية الهجينة مما ينعكس مباشرة على قدرة النظام على التوسع (Scalability) وتوزيع الأحمال، والحفاظ على مستويات مرتفعة من الاعتمادية والأمان.

يمتد التصميم بعد ذلك إلى الطبقات التفاعلية والتخزينية للأنظمة، حيث يتم صياغة تصميم واجهات المستخدم والتفاعل (UI/UX) استناداً إلى مبادئ التفاعل بين الإنسان والحاسوب لتسهيل حركة المستخدم داخل النظام وتقليل الأخطاء البشرية أثناء المدخلات. ويصاحب ذلك تصميم الواجهات البرمجية (APIs) لضمان التكامل السلس مع قواعد البيانات والأنظمة الخارجية. وفي الوقت نفسه، يُنفذ تصميم قواعد البيانات الفيزيائية من خلال تحويل مخططات البيانات المفهومية إلى جداول وقواعد بيانات منطقية وفيزيائية، مع تطبيق عمليات التطبيع (Normalization) لتقليل التكرار وتحسين كفاءة الاستعلامات وسرعة استرجاع البيانات. تُختتم هذه المرحلة بإعداد تصميم البرامج والمواصفات التفصيلية للموديولات البرمجية (Module & Program Specifications). يوثق هذا التصميم الخوارزميات الداخلية، وهياكل البيانات، وتدفق التحكم، وآليات معالجة الاستثناءات والأخطاء لكل وحدة برمجية، بحيث تصبح هذه المخططات بمثابة دليل تنفيذي ملزم ومكتمل للمطورين في مرحلة البناء.

التنفيذ implementation :

تمثل مرحلة التنفيذ الذروة العملية لدورة حياة النظام، حيث يتم تحويل المخططات والمواصفات المصممة إلى شفرات برمجية حية، وتجميعها وتجربتها، لنقل النظام من النطاق النظري إلى البيئة التشغيلية المباشرة. تبدأ هذه المرحلة بعملية البناء والتكوين البرمجي (Construction)، حيث يتولى المطورون كتابة الشفرات المصدرية استناداً إلى المواصفات التفصيلية للبرامج.
ويتزامن مع البناء تطبيق استراتيجية اختبار مكثفة متعددة المستويات، تبدأ باختبار الوحدات (Unit Testing) للتحقق من صحة الموديولات المنفصلة، ثم اختبار التكامل (Integration Testing) للتأكد من سلامة التفاعل بين الوحدات، تليها اختبارات النظام الشاملة (System Testing) لقياس الكفاءة والبيئة والأمان، وتُختتم باختبارات قبول المستخدم (User Acceptance Testing – UAT) لضمان تحقيق المتطلبات التشغيلية الفعلية.
بمجرد التحقق من جودة المنتج البرمجي، تنتقل العملية إلى مرحلة التركيب والتحويل (System Conversion)، وهي الخطوة الفاصلة للانتقال من النظام القديم إلى النظام الجديد داخل البيئة التنفيذية للمؤسسة[1]. وتعتمد المؤسسات في هذه الخطوة إحدى أربع إستراتيجيات تحويل رئيسية: التحويل المباشر الذي يتسم بالسرعة والانخفاض في التكلفة ولكنه يحمل مخاطر توقف عالية، أو التحويل الموازي الذي يضمن أقصى درجات الأمان عبر تشغيل النظامين بالتزامن ولكنه يستنزف الموارد المادية والبشرية، أو التحويل المرحلي الذي ينشر النظام عبر خطوات زمنية متدرجة، أو التحويل القيادي التجريبي الذي يقتصر على تطبيق النظام في موقع أو فرع محدد لقياس الأداء قبل التعميم[1]
تستلزم مرحلة التنفيذ تركيزاً مكثفاً على العوامل البشرية والتنظيمية من خلال إدارة التغيير والتدريب الشامل للمستخدمين وإعداد أدلة التشغيل والصيانة. ويساعد التدريب المنهجي على تذليل مقاومة التغيير وضمان الاستفادة الكاملة من القدرات التقنية للنظام. وبمجرد استقرار النظام في بيئة التشغيل الحية، يدخل النظام في حيز الدعم والصيانة المستمرة لمعالجة العيوب المكتشفة ميدانياً (Maintenance & Post–Implementation Support) ، وإجراء التحسينات الدورية لمواكبة التغيرات في بيئة الأعمال، وذلك وفق الأطر والأنشطة المحددة في معيار IEEE 1074 لإدارة دورة حياة البرمجيات.

2-7 إنشاء الجداول [15]:

[15] https://link.springer.com/article/10.1023/A:1006576209967

يُمثل إنشاء الجداول (Timetable Generation) العملية الحسابية التي تستهدف توزيع مجموعة من الأحداث والموارد البشرية والمكانية على فترات زمنية محددة دون إحداث أي تضارب تشغيلي و تُمثل الجدولة الأكاديمية في التعليم العالي أعلى درجات التعقيد في مسائل الأمثلة الحاسوبية، نظراً لتشابك المقررات، وتنوع المسارات، وتطبيق أطر المناهج المرنة القائمة على الساعات المعتمدة. يُصاغ نظام الجدولة كنموذج أمثلة رياضي يستهدف معالجة نوعين من القيود هما :

أ- القيود الصارمةHard Constraints

هي القيود التي يجب الالتزام بها حتى يكون الجدول صالحًا للتنفيذ. ومن أمثلتها:

- عدم تخصيص القاعة نفسها لمحاضرتين في الوقت نفسه.
- عدم وضع عضو هيئة تدريس في محاضرتين في الوقت نفسه.
- عدم وضع مجموعة طلاب في محاضرتين متزامنتين.
- ألا يتجاوز عدد الطلاب سعة القاعة.
- أن تتوافق تجهيزات القاعة مع متطلبات المقرر عند الحاجة.

وتؤكد الدراسات المتخصصة أن مخالفة القيود الصارمة تؤدي إلى جدول غير صالح، ولذلك يجب التأكد من تحقيقها أثناء عملية إنشاء الجدول.

ب- القيود المرنةSoft Constraints :

هي الشروط التي يُفضل تحقيقها لتحسين جودة الجدول، لكنها لا تمنع بالضرورة إنشاء الجدول إذا تعذر تحقيقها جميعًا. ومن أمثلتها:

- تقليل الفترات الزمنية الفارغة للطلاب.
- توزيع المحاضرات بصورة أفضل خلال أيام الأسبوع.

- تجنب وضع عدد كبير من المحاضرات المتتالية.
- مراعاة بعض تفضيلات أعضاء هيئة التدريس.
- تحسين استغلال القاعات.

ويُنظر إلى تحقيق أكبر عدد ممكن من هذه القيود على أنه وسيلة لتحسين جودة النتائج.

2-8 تقنيات ولغات البرمجه [16][17]:

تُعدّ تقنيات ولغات البرمجة من العناصر الأساسية في مجال تطوير البرمجيات ونظم المعلومات، تُستخدم في تصميم البرامج والتطبيقات وتنفيذها بما يتوافق مع متطلبات المستخدمين واحتياجات المؤسسات. ويمكن تصنيف لغات البرمجة وفقاً لعدة معايير، من أهمها مستوى اللغة، وطريقة بناء البرامج، ومجالات استخدامها.

16

https://www.bing.com/ck/a?!&&p=268996c4426477dff1f0c8a4ce11f099fe00bf45584b2ac654e1189f5d84097cJmltdHM9MTc5MDM4MDgwMA&ptn=3&ver=2&hsh=4&fclid=388e4801-a582-6448-1452-5fc0a44c65da&psq=%d8%a3%d9%85%d8%a7+%d8%aa%d9%82%d9%86%d9%8a%d8%a7%d8%aa+%d8%a7%d9%84%d8%a8%d8%b1%d9%85%d8%ac%d8%a9+%d9%81%d8%aa%d8%b4%d9%8a%d8%b1+%d8%a5%d9%84%d9%89+%d8%a7%d9%84%d8%a3%d8%b3%d8%a7%d9%84%d9%8a%d8%a8+%d9%88%d8%a7%d9%84%d9%85%d9%86%d8%a7%d9%87%d8%ac+%d9%88%d8%a7%d9%84%d8%a3%d8%af%d9%88%d8%a7%d8%aa+%d8%a7%d9%84%d9%85%d8%b3%d8%aa%d8%ae%d8%af%d9%85%d8%a9+%d9%81%d9%8a+%d8%b9%d9%85%d9%84%d9%8a%d8%a9+%d8%aa%d8%b7%d9%88%d9%8a%d8%b1+%d8%a7%d9%84%d8%a8%d8%b1%d9%85%d8%ac%d9%8a%d8%a7%d8%aa%d8%8c+&u=a1aHR0cHM6Ly9hci53aWtpcGVkaWEub3JnL3dpa2kvJUQ4JUFBJUQ4JUI3JUQ5JTg4JUQ5JThBJUQ4JUIxXyVEOCVBOCVEOCVCMSVEOSU4NSVEOCVBQyVEOSU4QSVEOCVBNyVEOCVBQQ

17https://www.bing.com/ck/a?!&&p=7c9007cfa7515657f56133856896a51e1929f5fd6d2a7b4eb0cbae2c989fe2f5JmltdHM9MTc5MDM4MDgwMA&ptn=3&ver=2&hsh=4&fclid=388e4801-a582-6448-1452-5fc0a44c65da&psq=%d8%aa%d9%82%d9%86%d9%8a%d8%a7%d8%aa+%d9%88%d9%84%d8%ba%d8%a7%d8%aa+%d8%a7%d9%84%d8%a8%d8%b1%d9%85%d8%ac%d9%87&u=a1aHR0cHM6Ly93d3cubWRyc2NlbnRlci5jb20vJUQ5JTg0JUQ4JUJBJUQ4JUE3JUQ4JUFBLSVEOCVBNyVEOSU4NCVEOCVBOCVEOCVCMSVEOSU4NSVEOCVBQyVEOCVBOS1wcm9ncmFtbWluZy1sYW5ndWFnZXMv

لغات البرمجة هي مجموعة من الأوامر والقواعد البرمجية التي تُمكّن المبرمج من توجيه الحاسوب لتنفيذ مهام محددة. ومن أبرز لغات البرمجة : C، C++، Java، Python، JavaScript، PHP، وتختلف كل لغة في خصائصها ومجالات استخدامها.

أما تقنيات البرمجة فتشير إلى الأساليب والمناهج والأدوات المستخدمة في عملية تطوير البرمجيات، ومن أبرزها البرمجة الإجرائية(Procedural Programming) ، والبرمجة الكائنية التوجه–(Object Oriented Programming)، والبرمجة الوظيفية(Functional Programming) ، بالإضافة إلى تقنيات تطوير تطبيقات الويب وتطبيقات الهواتف وقواعد البيانات.

وبذلك، تمثل لغات البرمجة الوسيلة التي يتم من خلالها كتابة التعليمات البرمجية، بينما تمثل تقنيات البرمجة الأساليب والمناهج التي تساعد في تنظيم هذه التعليمات وتطوير أنظمة برمجية ذات كفاءة وقابلية للصيانة والتطوير.

1.لغة Go[18] :

تعتبر Go لغة برمجية مجمعة (Compiled) واستاتيكية النمط تم تطويرها داخل أروقة شركة Google لتوفير أداء مرتفع وتزامن عالي الكفاءة.

- التاريخ والمفهوم :أطلقت رسمياً عام 2009 لتجمع بين سرعة تنفيذ لغة C وبساطة اللغات النصية.
- الخصائص الفنية :تمتلك نظام خفيف الوزن لإدارة التزامن عبر بروتوكولات Goroutines وChannels، ومجمع قمامة (Garbage Collector) فائق السرعة.

[18] https://claudsoft.com/blog/%D9%84%D8%BA%D8%A9-Go-%D8%AF%D9%84%D9%8A%D9%84-%D8%B4%D8%A7%D9%85%D9%84-%D9%84%D9%84%D9%85%D8%A8%D8%AA%D8%AF%D8%A6-%D9%88%D8%A7%D9%84%D9%85%D8%AD%D8%AA%D8%B1%D9%81-%D9%88%D8%AA%D8%B7%D8%A8%D9%8A%D9%82%D8%A7%D8%AA%D9%87%D8%A7-%D8%A7%D9%84%D8%B9%D9%85%D9%84%D9%8A%D8%A9

- الدور في النظام :تُسند إلى Go عملية تنفيذ محرك الجدولة التلقائي ومعالجة الخوارزميات الجينية والتحسين التوافقي .وتتيح قدرتها التزامنية اختبار آلاف الاحتمالات الجدولية في ثوانٍ معدودة دون تعطيل واجهات النظام الرئيسية.

2. لغة PHP [19]:

لغة نصية تعمل من جانب الخادم (Server-side) وتعد الحجر الأساس لبناء تطبيقات الويب الديناميكية.

- التاريخ والمفهوم :ابتكرها راسموس ليدروف عام 1995، وتطورت عبر محرك Zend Engineوصولاً إلى الإصدارات الحديثة ذات الأداء المرتفع.
- الخصائص الفنية :مرونة عالية، توافق تام مع مختلف خوادم الويب) مثل Apache و(Nginx، وتوفر مكتبات واسعة للاتصال بقواعد البيانات.
- الدور في النظام :تشكل لغة PHP اللغة الأساسية لبناء منطق الأعمال الخلفي (Backend Logic)وخدمات RESTful APIs لتسهيل تبادل البيانات وتطبيق سياسات الأمان والحجز.

3. لغةJavaScript [20]:

لغة برمجية عالية المستوى، ديناميكية، وتعمل في بيئات العميل (Client-side) والخادم (Server-side via Node.js).

- التاريخ والمفهوم :طورها بريندان إيتش في شركة Netscape عام 1995 تحت اسم LiveScriptقبل تغيير اسمها إلىJavaScript ، وتدير معاييرها منظمة Ecma International.

[19] https://www.ifhmsah.com/php
[20] https://web.dev/javascript?hl=ar

- الخصائص الفنية :معالجة لا تزامنية من خلال محركات الأحداث(Event-driven) ، دعم مرن للكائنات والبرمجة الوظيفية.
- الدور في النظام :إدارة الشاشة التفاعلية للتقويم الأكاديمي، وتحديث واجهات حجز القاعات بلحظيتها دون الحاجة لإعادة تحميل الصفحات، فضلاً عن معالجة المدخلات فورياً.

4. لغة Python [21]:

لغة برمجة مفسرة، عالية المستوى، وتتسم بقراءة كود سلسة وبنية نصوص واضحة.

- التاريخ والمفهوم :طورها خايدو فان روسم عام 1991، وازدادت شهرتها الأكاديمية مع إطلاق إصداراتها المتتالية.
- الخصائص الفنية :دعم أنماط برمجية متعددة (كائنية، وظيفية، إجرائية)، واحتواؤها على مكتبات رياضية وإحصائية ضخمة مثل NumPy و Pandasو.DEAP
- الدور في النظام :تُستخدم كأداة رديفة لتحليل بيانات الاستغلال المكاني للقاعات، وتحديد الأنماط الزمنية للحجوزات، واختبار الخوارزميات الذكية بشكل أولي.

5.لغة Java [22]:

لغة كائنية التوجه بالكامل(Fully Object-Oriented) ، مصممة لتعمل عبر مبدأ "اكتب مرة واحدة، شغل في أي مكان.(WORA) "

- التاريخ والمفهوم :طوّرها جيمس غوسلينغ في شركة Sun Microsystems عام 1995 وتديرها حالياً شركة.Oracle

[21] https://developers.google.com/edu/python/introduction?hl=ar
[22] https://alriyadaaltaqniya.blogspot.com/2026/07/What-is-Java-A-comprehensive-explanation-for-beginners-and-its-most-important-uses-in-application-development.html

- الخصائص الفنية :تعتمد على بيئة آلة جافا الافتراضية(JVM) ، وتوفر إدارة آمنة للذاكرة، ودعماً قوياً للتطبيقات السحابية وتعدد المسارات.(Multithreading)
- الدور في النظام :تستخدم لبناء الخدمات الميكروية (Microservices) المؤسسية للربط بين نظام الجدولة وأنظمة إدارة التعلم (LMS) وأنظمة القبول والتسجيل بإنتربرايز الجامعة.

6.لغةC++[23] :

لغة برمجة مجمعة، عالية الأداء، تجمع بين خصائص لغات المستوى العالي والمستوى المنخفض.

- التاريخ والمفهوم :طورها بيارن ستروستريب في مختبرات بيل كتطوير للغة Cمع إضافة البرمجة الكائنية.
- الخصائص الفنية :التحكم المباشر بالذاكرة عبر المؤشرات(Pointers) ، السرعة الفائقة في تنفيذ التعليمات الحسابية، واستغلال موارد المعالج بكفاءة مثالية.
- الدور في النظام :تُستخدم لبناء وحدات المحاكاة عالية السرعة والمهام الحسابية الشاقة المعنية بحل المصفوفات الضخمة المعقدة للجدولة عند استهلاك مئات المقررات والقاعات.

7. لغةHTML5 [24]:

[23] https://halabtech.com/%D8%B4%D8%B1%D8%AD-%D9%85%D8%A8%D8%B3%D8%B7-%D8%B9%D9%86-%D9%84%D8%BA%D8%A9-c-%D8%A7%D9%84%D8%B3%D9%8A-%D8%A8%D9%84%D8%B3-%D8%A8%D9%84%D8%B3/

[24] https://multaqaweb.com/blog/what-is-html

اللغة الهيكلية القياسية لبناء وتنظيم محتوى مستندات صفحات الويب.

- التاريخ والمفهوم :طورها السير تيم بيرنرز لي عام 1993، وتطورت عبر منظمات W3C و WHATWGحتى الوصول للنسخة المستقرة.HTML5
- الخصائص الفنية :دعم الوسوم الدلالية(Semantic Tags) ، توفير عناصر مدخلات متقدمة، والتكامل الأصيل مع الوسائط المتعددة دون إضافات خارجية.
- الدور في النظام :بناء البنية الهيكلية لصفحات حجز القاعات، وعرض الجداول، ونماذج إدخال المقررات والقيود الأكاديمية.

8.لغه [25] css3 :

لغة توصيفية مخصصة للتحكم في مظهر وتنسيق المستندات المكتوبة بلغة.HTML

- التاريخ والمفهوم :أطلقتها W3C عام 1996 لإعادة فصل التنسيق الجمالي عن البنية الهيكلية، وتعد CSS3 الإصدار المعياري الحديث.
- الخصائص الفنية :تقنيات التخطيط المرن(Flexbox & CSS Grid) ، التحويلات والتأثيرات الحركية، ودعم استجابة الشاشات Responsive) Design).
- الدور في النظام :تصميم واجهات مستخدم متناسقة وعصرية تنظم مصفوفة الجدول الأسبوعي للقاعات، وتبرز حالات التعارض بالألوان والتصميمات المريحة للبصر.

[25] https://www.arageek.com/l/%d9%85%d8%a7-%d9%87%d9%8a-%d9%84%d8%ba%d8%a9-css

9.لغه [26] SQL :

SQLهي اللغة القياسية للتعامل مع قواعد البيانات العلائقية، بينما تمثل MySQL النظام الإداري المفتوح المصدر للتحكم بتلك البيانات.

- التاريخ والمفهوم :أنشئت شركة MySQL AB السويدية نظامMySQL ، المملوك حالياً لشركةOracle ، لتقديم قواعد بيانات سريعة وآمنة.
- الخصائص الفنية :دعم كامل لخصائص ACID لضمان سلامة المعاملات، ودعم المفاتيح الأجنبية والربط العلائقي، والسرعة الفائقة في معالجة الاستعلامات الضخمة.
- الدور في النظام :تخزين واسترجاع بيانات الكليات، المقررات، أعضاء هيئة التدريس، القاعات وسعاتها، سجلات الحجز المنجزة، والجداول الصادرة.

10. البيئة البرمجية المتكاملة والأطر الحديثة(React, Laravel, Bootstrap)

مجموعة من أطر العمل والمكتبات المتكاملة التي تسرع وتوحد بناء النظام:

- **React**[27]: مكتبة JavaScript مطورة من Facebook تعتمد على نظام المكونات المعزولة (Components) والشجرة الافتراضية (Virtual DOM) لبناء واجهات أمامية سريعة.
- **Laravel**[28]:إطار عمل PHP متميز يطبق نمط النمذجة والواجهة والمتحكم (MVC)، يوفر أدوات أمان متقدمة، وأداة Eloquent ORM للتعامل مع قاعدة البيانات بسهولة.

---

[26] https://academy.hsoub.com/programming/sql/%D8%A3%D8%B3%D8%A7%D8%B3%D9%8A%D9%91%D8%A7%D8%AA-%D9%84%D8%BA%D8%A9-sql-r585/

[27] https://www.how-ar.com/how-to-develop-web-applications-using-react-framework/#google_vignette

[28] https://blog.naqrah.net/ar/what-is-laravel/81#google_vignette

- **Bootstrap**[29]: إطار عمل CSS/JS مفتوح المصدر أنشئ بواسطة Twitterلتوفير عناصر واجهة جاهزة ومتجاوبة تدعم العرض المالي الذكي عبر مختلف الأجهزة.

[29] https://wiki.hsoub.com/Bootstrap